

An encrypted traffic identification method based on multi-scale feature fusion

Peng Zhu^a, Gang Wang^{a,*}, Jingheng He^a, Yueli Dong^b, Yu Chang^a

^a College of Data Science and Application, Inner Mongolia University of Technology, Hohhot, China

^b Ordos Institute of Applied Technology Big Aircraft College, Ordos, China

ARTICLE INFO

Keywords:

Encrypted traffic classification
Global optimal solution
Multi-scale feature fusion
ET-BERT
One-dimensional convolutional neural network

ABSTRACT

As data privacy issues become more and more sensitive, increasing numbers of websites usually encrypt traffic when transmitting it. This method can largely protect privacy, but it also brings a huge challenge. Aiming at the problem that encrypted traffic classification makes it difficult to obtain a global optimal solution, this paper proposes an encrypted traffic identification model called the ET-BERT and 1D-CNN fusion network (BCFNet), based on multi-scale feature fusion. This method combines feature learning with classification tasks, unified into an end-to-end model. The local features of encrypted traffic extracted based on the improved Inception one-dimensional convolutional neural network structure are fused with the global features extracted by the ET-BERT model. The one-dimensional convolutional neural network is more suitable for the encrypted traffic of a one-dimensional sequence than the commonly used two-dimensional convolutional neural network. The proposed model can learn the nonlinear relationship between the input data and the expected label and obtain the global optimal solution with a greater probability. This paper verifies the ISCX VPN-nonVPN dataset and compares the results of the BCFNet model with the other five baseline models on accuracy, precision, recall, and F1 indicators. The experimental results demonstrate that the BCFNet model has a greater overall effect than the other five models. *Its accuracy can reach 98.88%.*

1. Introduction

During the past decade, network traffic has experienced exponential growth with the emergence and expansion of new network technologies, such as the Internet of Things (IoT) [1], big data, and cloud computing. Monitoring and categorizing network traffic is of great significance in network QoS control (QoS) and network resource utilization planning [2]. With the increase in data privacy awareness, many websites have begun to use encryption for data transmission, creating obstacles to network traffic management. Owing to the increasing complexity and diversity of encrypted traffic, the difficulty of traffic identification using network detection systems has increased significantly. Therefore, without decrypting the encrypted traffic, the accurate classification of encrypted traffic has gradually become a research hotspot.

Network-traffic classification relied heavily on port numbers in the early days of the Internet. The port-number-based approach uses port communication in TCP/UDP, and this classification method is used to map and match the specified ports to application protocols. For example, port 22 represents the SSH application protocol, and port 53 represents

the Domain Name Service (DNS) application protocol. As Internet technology has evolved, its accuracy has declined, as new applications either use well-known port numbers to mask traffic or avoid using standard registered port numbers. Moore et al. [3] conducted experiments, analyzed the results, and found that the accuracy of the port number-based approach in the field of traffic classification was only 69%. Traffic types have become increasingly complex, and the limitations of port-based traffic classification methods have become increasingly evident. Deep Packet Inspection (DPI) [4] is the next generation of traffic classifiers that analyze packets to find patterns or keywords [5]. This was performed by matching the packet content with predefined strings for each traffic type to achieve the classification. DPI techniques are used to compare network traffic packets with the data feature library using single pattern-matching algorithms (e.g., the KMP [6], BM [7], and AC algorithms [8]). Despite the many techniques proposed to increase payload inspection efficiency, encrypted network traffic remains an issue that cannot be resolved because the plaintext data in traffic encrypted with a key become sparse or fuzzy, making the construction of data feature libraries difficult. Several subsequent studies have utilized

* Corresponding author.

E-mail address: wg@imut.edu.cn (G. Wang).

<https://doi.org/10.1016/j.array.2024.100338>

Received 17 August 2023; Received in revised form 17 February 2024; Accepted 18 February 2024

Available online 22 February 2024

2590-0056/© 2024 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

the statistical features of encrypted traffic to classify traffic, as well as classical machine learning algorithms that have performed well across the board in various domains to recognize encrypted traffic without plaintext. Researchers have applied machine learning algorithms such as plain Bayes, Markov chains, K-means [9], and linear discriminant analysis with KNN [10] to classify encrypted traffic. However, machine learning requires the researcher to construct feature sets manually based on the expertise of experts, and the quality of the feature set significantly affects the results of the classification task [11].

To resolve this technical bottleneck, deep learning has been rapidly developed in the field of encrypted traffic identification. Artificial neural networks are ideal for deep-learning techniques in feature engineering because they can mine features on their own, significantly reduce human intervention, and discover associations between data that humans have not yet discovered. Chen et al. [12] proposed a deep learning method for recognition using a convolutional neural network (CNN), which utilizes a CNN to automatically extract features from the original data and then classifies them by training a learning classifier. It can recognize a target more accurately than traditional machine learning methods and does not require human feature selection. In a study of traffic features, it was found that traffic has temporal and spatial features similar to those of text. Chen et al. [13] proposed an attention-CNN-based encrypted traffic recognition method based on identifying the initial features of encrypted traffic data and compressing the data into 1D-CNN models for further extraction of temporal and spatial features and features for feature compression and further extraction. Liu et al. [14] proposed a stream sequence network based on stack autoencoders (SAE) to classify encrypted traffic using recurrent neural networks (RNN). Zouyuan et al. [15] proposed a long short-term memory (LSTM)-based method for detecting malicious network traffic in encrypted environments and used LSTM to analyze the long-distance dependencies of traffic transmission patterns to classify encrypted traffic. Wang et al. [16] developed a classifier that processes traffic data into images. A CNN was used to classify malware traffic based on these images. The experimental results showed that 1D-CNNs performed better than 2D-CNNs in traffic classification. Wang et al. [17] proposed an encrypted traffic classification method that resists grayscale graphs against attacks in traffic classification, constructs a topology graph by extracting traffic interaction information, such as packet load length, packet sequence, direction, and clusters, and classifies encrypted traffic using a GCN-based [18] classification model. Yao et al. [19] analyzed the network traffic time series modeled with RNN, and in addition, they also introduced an attention mechanism to assist in network traffic classification, i.e., LSTM based on attention mechanisms. Experiments were conducted on the ISCX VPN-nonVPN dataset to classify 12 types of encrypted application traffic, and the proposed method achieved 91.2% accuracy. Kim et al. [20] proposed models of convolutional neural networks and LSTM (C-LSTM). By using the packets in the encrypted traffic as a one-dimensional grayscale image, the neural network can learn the features independently of the network, which are then passed on to the CNN model for learning, where they are converted into vectors. To obtain the classification results, a sequence-learning algorithm was used to learn the encryption using the LSTM model, which was finally passed to the network of fully connected nodes.

In many current encrypted traffic detection methods, most deep-learning-based models use CNN to learn traffic features for classification. However, ordinary CNN has limitations in feature extraction. Traditional CNN structures use a fixed-size convolutional kernel and have a relatively small number of convolutional layers, making them prone to losing key information when processing traffic data. However, CNN only learns local features in the data streams, ignoring the relative positions between strings and the relationship between local features as a whole.

To address these issues, this study designed a feature fusion-based classification method for encrypted traffic using the BCFNet model. First, the local spatial features of encrypted traffic byte data are captured

using an improved Inception one-dimensional convolutional network structure. Second, the Masked BURST Model (MBM) captures the same relationship between distinct datagram bytes in a BURST and represents them through their contexts by obtaining the bi-directional features of the encrypted traffic using the ET-BERT model and modeling the transmission relationship between previous and subsequent BURSTs using the Same-origin BURST Prediction (SBP) method, which is used to improve the accuracy and generalization ability for classification in complex network environments. Several state-of-the-art baseline models for classified encrypted traffic were outperformed by a network framework comprising inception 1D convolution and ET-BERT.

2. BCFNet method for multi-scale feature fusion

2.1. ET-BERT

The core structure of ET-BERT [21] is a multilayer transformer encoder. Each encoder consists of multiple sublayers, including multi-head self-attention [22], a feed-forward network, and a normalization layer. Multihead Attention is at the heart of the transformer, which is a model that focuses on information from multiple locations in parallel; thus, global dependencies may be better captured. If the input traffic sequence is $X = \{X_1, X_2, \dots, X_n\}$, where n is the length of the traffic sequence and the dimension of each vector is d , h attention heads are used to compute the multihead attention, each of which generates a query vector q_i , key vector k_i and v_i , all of which are obtained by linearly transforming the input traffic sequence. We multiply the input traffic sequence using three weight matrices:

$$q_i = XW_q^{(i)} \quad (1)$$

$$k_i = XW_k^{(i)} \quad (2)$$

$$v_i = XW_v^{(i)} \quad (3)$$

For each attention head, we first calculate the attention weights α_i and then use these weights to average the weight of the value vectors V_i to obtain the attention z_i , that is,

$$\alpha_i = \text{soft max} \left(\frac{q_i k_i^T}{\sqrt{d_k}} \right) \quad (4)$$

$$z_i = \sum_{j=1}^n \alpha_{ij} v_{ij} \quad (5)$$

Finally, the attentional representations of each attentional head are spliced together to form the final multi-head attentional representation, that is,

$$Z = \text{concat}(z_1, z_2, \dots, z_h) W_O \quad (6)$$

The multi-head attention mechanism maps the input traffic sequence to multiple subspaces, which better captures the relationships between the different parts of the traffic sequence. It simultaneously observes the input traffic sequence from different perspectives and granularities to capture global features.

To convert traffic into similar text in a natural language, byte traffic is converted into a token of the BURST structure, which originates from a request in a single-session flow or a corresponding set of temporally adjacent network packets. The BURST is defined as

$$\text{BURST} = \begin{cases} B^{\text{src}} = \{p_m^{\text{src}}, m \in N^+\} \\ B^{\text{dst}} = \{p_n^{\text{src}}, n \in N^+\} \end{cases} \quad (7)$$

The numbers of source-to-target and target-to-source unidirectional packets are denoted by m and n , respectively, and the session flow consisting of packets is identified by a quintuple.

The model predicts the task using the mask and homology BURST models to mine and characterize the implicit correlation information of

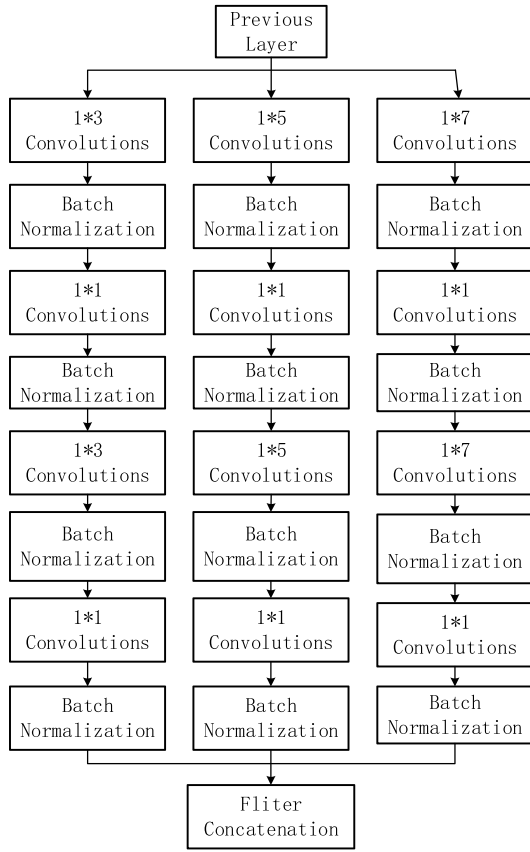


Fig. 1. Improved Inception one-dimensional convolution module.

encrypted traffic messages.

ET-BERT is trained to predict tokens within mask locations based on context, and we use this ability to mask k tokens randomly for the input sequence X because the mask tokens are replaced by special tokens [MASK]. The loss function is formally defined as a negative log-likelihood:

$$L_{MBM} = - \sum_{i=1}^k \log(P(MASK_i = token_i | \bar{X}; \theta)) \quad (8)$$

The homologous BURST prediction task learns the dependencies between packets within BURST. The purpose of each is to determine whether two BURSTs originate from the same BURST by dividing it into two subparts. Specifically, sub-BURST pairs (0 for paired sub-BURSTs and 1 for unpaired sub-BURSTs) were used.

$$L_{SBP} = - \sum_{j=1}^n \log(P(y_j | B_j; \theta)) \quad (9)$$

Consequently, the final training objective is the sum of the two losses, defined as follows:

$$L = L_{MBM} + L_{SBP} \quad (10)$$

The key to the preprocessing phase is to extract raw traffic using BURST structures with content transmission characteristics and significantly biased data. To exploit the distinguishable characteristics of traffic information, mask prediction is learned for symbol contexts, whereas BURST structures are truncated into pairs and predicted as BURST subpairs that originate from the same BURST. Both tasks can consider the content relevance of traffic and capture global traffic features better than learning only the relationships between symbol contexts.

The token, position, and segment embeddings constitute the input

form of the model.

Token embedding: Tokens are represented using byte encoding by reading the raw preprocessed traffic data, where each token cell ranges from zero to 65535. A token termed [CLS] is used as the starting point for each sequence, and its output is used to represent the entire traffic sequence. Data with a length less than the minimum requirement were padded with a token [PAD], and data with a length greater than the minimum were padded with a token [SEP].

Positional Embedding: Positional embedding is used to encode flow data in a manner that captures the relative positions of byte flows in relation to the flow transmission.

Split Embedding: it is divided into two parts, Sub-BURST A and Sub-BURST B. Embedding a flow is a unique encoding process that captures the relative position information between traffic bytes.

2.2. One-dimensional convolutional module based on multi-scale fusion

In this section, we propose a robust method for classifying encrypted traffic using multiscale feature fusion convolutional neural networks. To improve the generalization ability of the model, a normalization layer and an improved inception module were introduced in addition to the traditional convolutional and fully connected CNN layers.

First, the model accepts network traffic data to be processed, which are composed of byte streams that are essentially one-dimensional data. Thus, the one-dimensional convolution designed in this study consists of multiple one-dimensional convolution modules. The encrypted traffic is processed with features and then fed into a one-dimensional convolution in the form of byte vectors. The first layer of the module has one channel and consists of convolution layers with convolution kernel sizes of 1×3 , 1×5 , and 1×7 . This convolution is expressed as follows:

$$M^j = f \left(\sum_i w^{ij} * x^i + b^j \right) \quad (11)$$

M^j and x^i represent the j th and i th output mappings, respectively. w^{ij} represents the convolution filter weights. $*$ is the j th mapping of b^j , which is characterized by convolution and represents the bias parameter of the mapping. This function represents the activation function. A rectified linear unit (ReLU) function was used to protect the model against overfitting. The activation function is expressed as

$$f(x) = \begin{cases} x & \text{if } f(x) > 0 \\ 0 & \text{if } f(x) \leq 0 \end{cases} \quad (12)$$

In backpropagation, the derivative of the ReLU activation function is 1 when the input is positive and 0 when the input is negative, which means that the gradient will not be narrowed, thus avoiding the problem of a vanishing gradient and accelerating the training of the model. A negative input results in an activation function output of zero. This sparse activation feature can result in sparse neurons in the neural network, which can reduce the occurrence of overfitting and improve generalization ability.

Traditional convolutional neural networks can form a complex nonlinear model by superimposing multiple convolutional layers. However, all convolutional kernels in the same layer have the same hyperparameters, which does not allow good extraction of features of different sizes. Therefore, the concept of an inception module was introduced to use convolutional kernels of different sizes to extract relevant features and fuse the features of different sizes to provide a basis for classification judgment. This can be seen in Fig. 1, the first layer of this model consists of convolutional layers with convolutional kernel sizes of 1×3 , 1×5 , and 1×7 , and the number of its output channels is 256. The second layer converts the feature maps of 256 channels into feature maps of 512 channels by adding 1×1 point convolution, which improves the expressive ability of the model by increasing the number of channels in the feature maps. There were 128 channels in both the third and

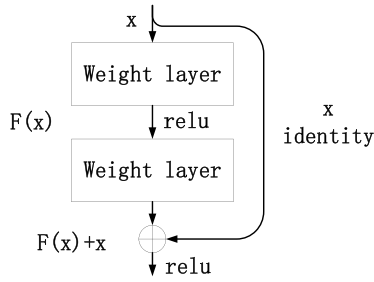


Fig. 2. Residual structure.

fourth layers. Finally, highly correlated features are clustered together for multiscale feature extraction to form a new feature set.

Increasing the depth of a neural network may lead to overfitting and covariance bias. To address these problems, this study introduced a batch normalization layer after a convolutional layer. Using batch normalization, the inputs of each layer during training were adjusted to the same distribution to prevent overfitting and covariance bias. Batch normalization [23] was calculated as follows:

$$y = \frac{\gamma}{\sqrt{\text{Var}[X] + \epsilon}} + \left(\beta - \frac{\gamma E[X]}{\sqrt{\text{Var}[X] + \epsilon}} \right) \quad (13)$$

2.3. BCFNet model structure

In this study, one-dimensional convolutional features were fused with the features extracted from ET-BERT, and a model for classifying encrypted traffic using BCFNet (the features of ET-BERT and the 1D-CNN Fusion Network) was proposed. The local features of the encrypted traffic were extracted more comprehensively using the improved inception one-dimensional convolution module, and the global features of the encrypted traffic were extracted by capturing the relationship between the bytes of different packets using ET-BERT. The flow of the model is illustrated in Fig. 3. First, processing of the original data was performed, followed by processing in the ET-BERT and 1D-CNN models, and the ET-BERT model required the input data to be processed into a

token form using the Datagram2Token tool. In this study, a 1D-CNN branch was introduced to extract features. In the 1D-CNN model, the design of the 1D convolutional modules is inspired by the inception structure. Each module has three feature-extraction branches that use different convolutional kernels for feature processing. Finally, the features of the three branches were spliced. A residual structure [24] was introduced between the second and third modules, where the first module outputs features as x , the second module outputs features as $F(x)$, and after the residual structure, the resulting features are called $F(x) + x$. Finally, all the outputs of the third layer were weighted in the fourth module, and feature fusion was performed for mean smoothing over its feature map.

A residual structure was proposed to solve the gradient problem arising from an increase in the number of network layers. As shown in Fig. 2, an encrypted signal is formally represented using x . After x passes through the convolutional layer and activation function, the output is $F(x)$, where $F(-)$ denotes the signal processing mode. By introducing a residual structure, constant mapping can be performed on the encrypted signal x . Superimposing the constant mapping and convolution operations yields $F(x) + x$. The constant mapping operation is simple to perform and does not increase the computational complexity, and the operation flow is shown in Fig. 2.

In the ordinary network structure, the weights are stacked up with the purpose of fitting features. But such a relationship is poorly fitted for various reasons, so by constant mapping, new features can be fitted when the residuals are not zero.

The 1D-CNN vector feature vector was obtained after the 1D-CNN model processing, and the ET-BERT vector feature vector was obtained after the ET-BERT model processing, whose feature vector dimensions were 384 and 768, respectively. Subsequently, the two feature vectors are input into the fully connected layers to transform them into the same 256-dimensional vectors, and the outputs of these layers are transformed into the same 256-dimensional vectors using the tanh activation function to perform a nonlinear transformation to produce a better feature representation. We defined two weight layers, etbert_weight and cnn_weight, to compute the weights of the ET-BERT and 1D-CNN in the fusion. The output of these layers is a scalar indicating the level of attention of the model for each feature vector. The

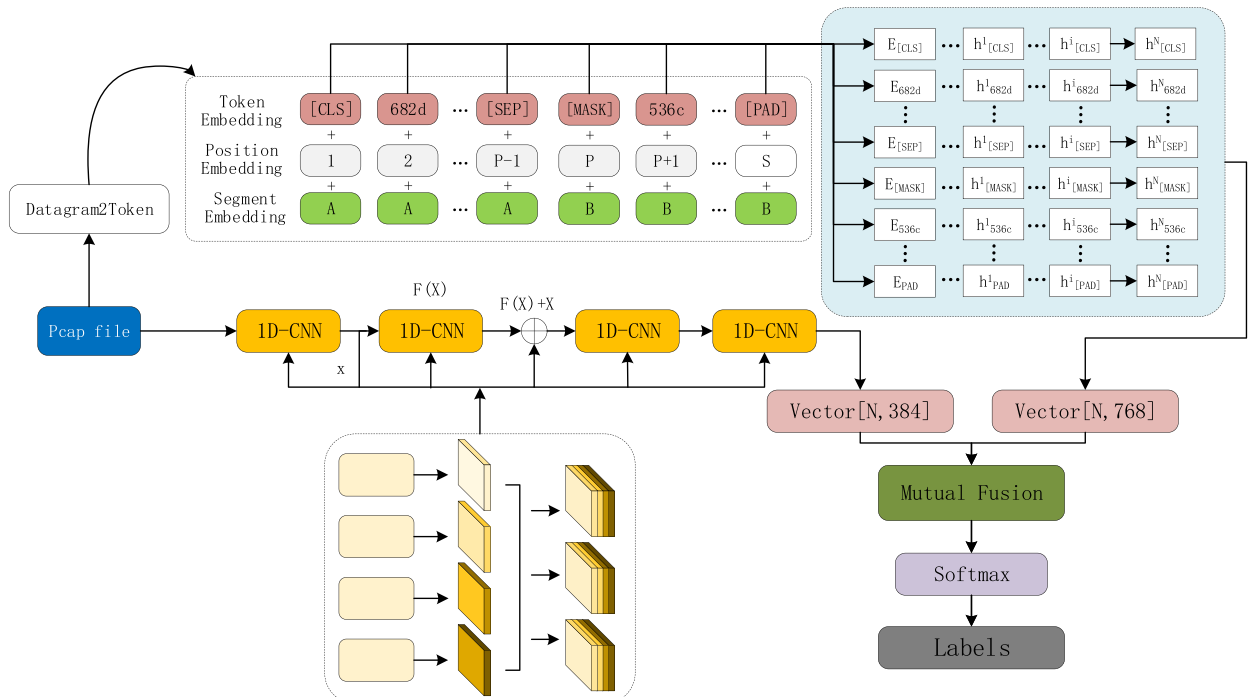


Fig. 3. BCFNet model structure based on multi-scale feature fusion.

Table 1
ISCX VPN-nonVPN dataset.

Experimental Scenario	Content	Number of Categories	Category Number	Specific Category
1	Protocol encapsulation and regular encrypted traffic identification	2	0 1	Protocol encapsulation Regular encrypted traffic identification
2	Regular encrypted traffic classification	6	0 1 2 3 4 5	Chat Email File P2P Streaming Voip
3	Protocol Encapsulation Traffic Classification	6	0 1 2 3 4	Chat-VPN Email-VPN File-VPN P2P-VPN Streaming-VPN
4	Encrypted Traffic Classification	12	5 0 1 2 3 4 5 6 7 8 9 10 11	Voip-VPN Chat Email FT P2P Streaming Chat-VPN Email-VPN FT-VPN P2P-VPN Streaming-VPN Voip-VPN

Table 2
ISCX VPN-Application dataset.

Type of flow	Marking content	Traffic content
Chat VPN-Chat	Traffic generated by instant messaging applications	Aim, Facebook, Hangout, ICQ, Skype
Email VPN-Email	Traffic generated by various types of emails	Email, Gmail
File VPN-File	Traffic generated by file transfers	FTPS, SCP, SFTP, Skype
Streaming VPN-Streaming	Multimedia application traffic	Netflix, Youtube, Spotify, vimeo
Voip VPN-Voip	Voice application traffic	Facebook, Hangouts, Skype, Buster
P2P VPN-P2P	File sharing protocol program traffic	Tor, Torrent

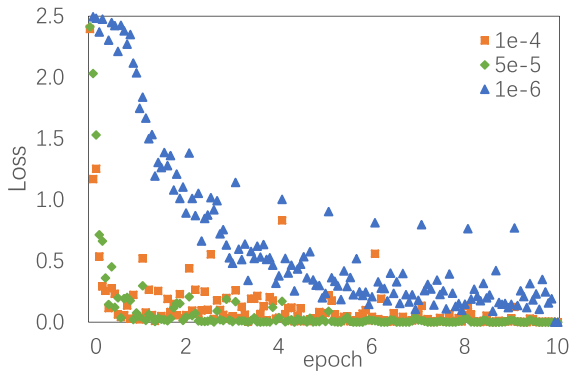


Fig. 4. The loss value of the model under different learning rates.

Table 3
Effect of packet sequence length on model accuracy.

Sequence length	Service	Application
8	65.68	92.70
16	98.18	99.21
32	98.68	99.29
64	98.88	99.44
128	98.79	99.37

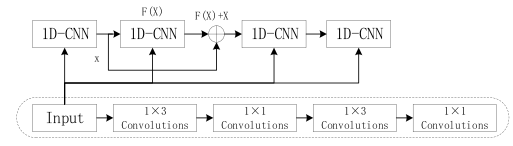


Fig. 5. MD1CNN Network Module structure.

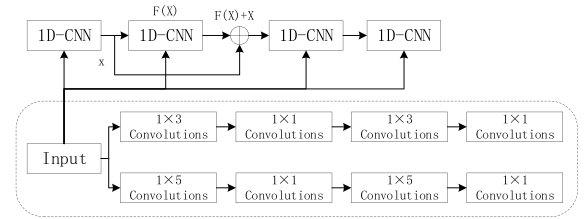


Fig. 6. MDT4CNN network module structure.

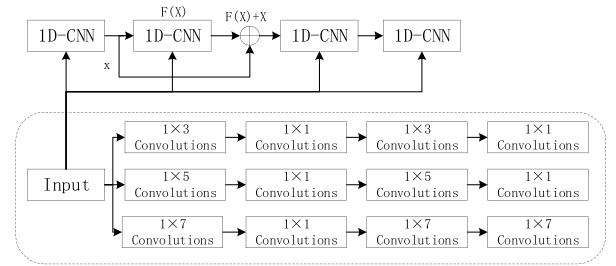


Fig. 7. IM-CNN network module structure.

Table 4
One-dimensional convolution module ablation experiment.

classification model	Internal structure of the module	Accuracy	Precision	Recall	F1-Score
MD1CNN	1×3 convolution	0.9695	0.9697	0.9695	0.9696
MD2CNN	1×5 convolution	0.9682	0.9684	0.9682	0.9683
MD3CNN	1×7 convolution	0.9669	0.9669	0.9668	0.9668
MDT4CNN	$1 \times 3 + 1 \times 5$ convolution	0.9743	0.9744	0.9743	0.9743
MDT5CNN	$1 \times 3 + 1 \times 7$ convolution	0.9721	0.9723	0.9720	0.9721
MDT6CNN	$1 \times 5 + 1 \times 7$ convolution	0.9719	0.9721	0.9719	0.9720
IM-CNN	$1 \times 3 + 1 \times 5 + 1 \times 7$ convolution	0.9766	0.9771	0.9766	0.9767

scalar of the output of the weight layer is obtained by passing it through another fully connected layer and performing a tanh activation function of the model. Finally, the output of each model is represented as a separate vector, the two vectors are spliced, and the scores are

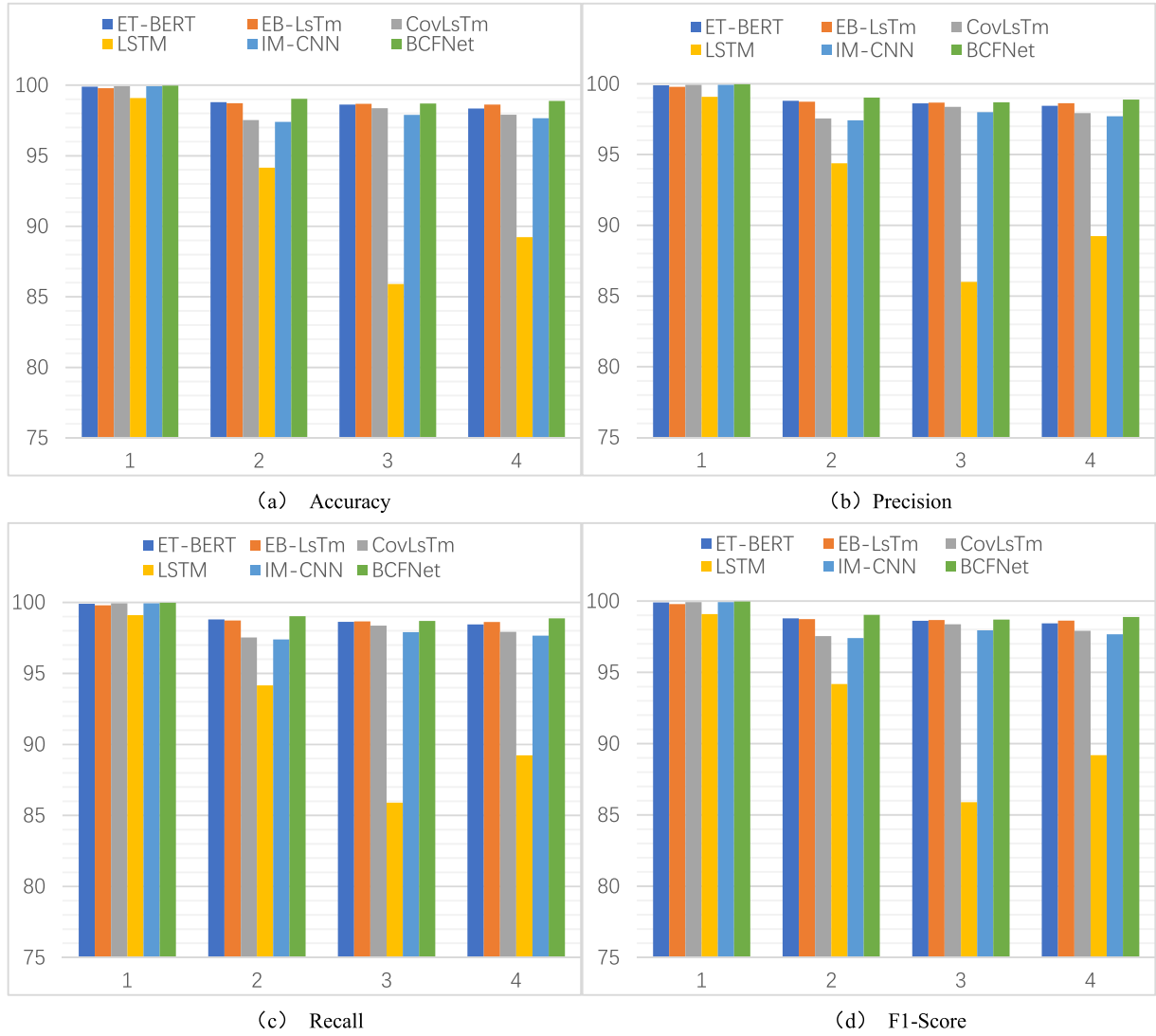


Fig. 8. ISCX VPN-nonVPN dataset classification performance indicators of each model in four scenarios.

normalized using the softmax function such that the weights sum to one. The softmax real vector is transformed into a vector of probability distributions, where each component represents the probability that the input belongs to each possible category, fitting the input vector to a real number within the range $[0, 1]$. Here, w_i with x is the column vector, and K is the number of categories in the observed sample, denoted as

$$p(i) = \frac{\exp(w_i x)}{\sum_{k=1}^K (w_k x)} \quad (14)$$

The two features are fused, and the weights of the two features are calculated and input into the fully connected layer for classification, where $\alpha_{ET-BERT}$ denotes the weights of the features extracted by the ET-BERT model and α_{IM-CNN} denotes the weights of the features extracted by the 1D-CNN model. The main features of this study are as follows:

$$f_{TOTAL} = \alpha_{ET-BERT} f_{ET-BERT} + \alpha_{IM-CNN} f_{IM-CNN} \quad (15)$$

3. Experimental results and analysis

3.1. Experimental environment

In this study, the experiments were developed based on the Pycharm 3.9 software platform using the PyTorch 1.10.1 framework to build the

network. The experimental environment was the Ubuntu 20.04 operating system, and the graphics card model was an NVIDIA GeForce RTX3090. The Adam optimizer was used in this study, and the learning rate was set to $5e-5$. When training the network, the loss function in this study is a categorical cross-entropy, which accelerates the convergence process and is calculated as follows:

$$Loss = - \sum \left\{ X_i^n \log_2 (Y_i^n) \right\} \quad (16)$$

In the set of samples, each sample has a label n ; X_i and Y_i are the measured and predicted values, respectively; and the measurement labels of the samples are given by a vector of unique heat codes.

3.2. Dataset

The dataset used in this study was the ISCX VPN-nonVPN [25], which was collected from the Canadian Institute for Cybersecurity Research. It is the most widely used raw traffic dataset in the field of encrypted traffic identification, with labels provided in the raw PCAP format. It consists of man-made traffic that contains different traffic types as well as information about the associated applications, collected both through regular sessions and through sessions encapsulated over a VPN.

Given this structure, tags (i.e., traffic types and applications) can be

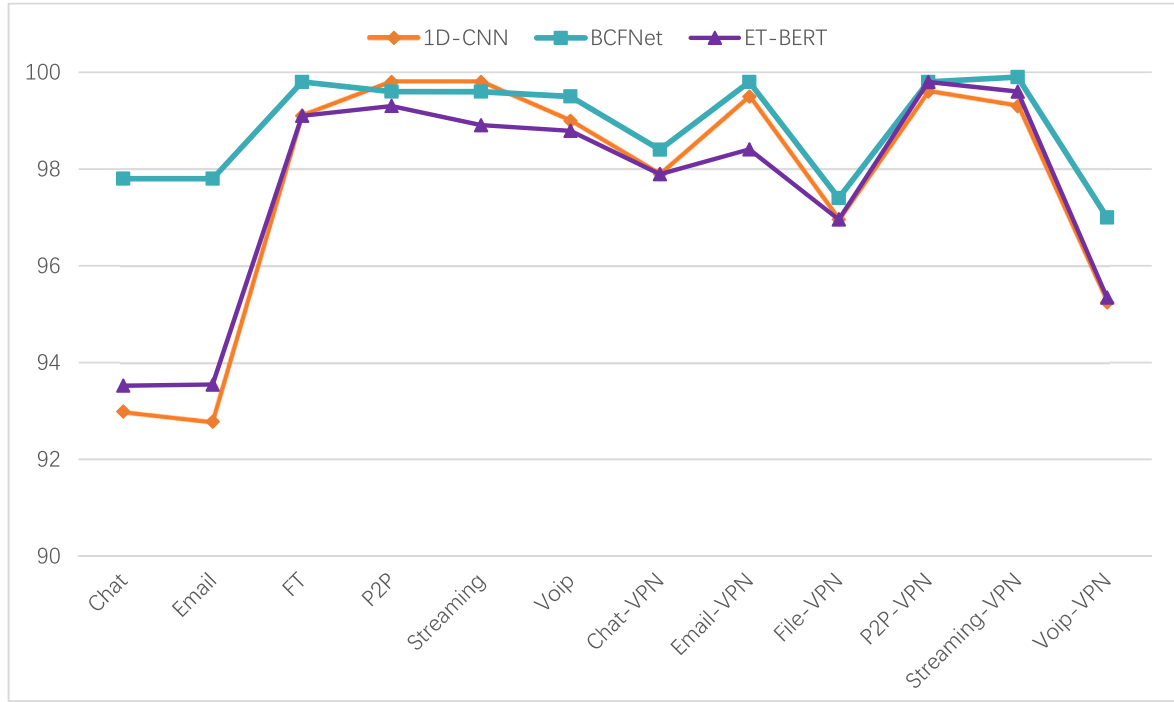


Fig. 9. Comparison chart of F1 values for ISCX VPN-nonVPN 12 class traffic.

associated with any segment of the original data. A dataset with seven types of regular encrypted traffic and seven types of protocol-encapsulated traffic is available, of which some files, such as a “Facebook_video_pcap,” can be categorized either as a “Browsing” or a “Streaming.” Although the problem only applies to “Streaming,” all other files related to “Browsing,” and “VPN-Browsing” also exhibit this condition. In this paper, the traffic categories “Browsing” and “VPN-Browsing” are removed, so a total of six major traffic categories are selected from the dataset: Email, P2P, Streaming, Voip, Chat, and Email. Therefore, 12 traffic categories were selected from the dataset: email, peer-to-peer (P2P), streaming, voice, chat, and email, which are the six major application types and their corresponding VPN traffic.

As shown in Table 1, this study sets up four experimental scenarios that were established based on the ISCX VPN-nonVPN (ISCX VPN-Service) dataset [26]. Scenario 1 is the VPN protocol encapsulation and regular traffic identification; Scenario 2 is the regular encrypted traffic classification, which is a six-classification problem; Scenario 3 is the VPN protocol encapsulation traffic classification, which is also a six-classification problem; and Scenario 4 involves all encrypted traffic in scenarios 2 and 3, which is a more complex 12-classification problem. We selected 6000 packets, totaling 72000 data points from each class, and selected 80% as the training set, 10% as the test set, and 10% as the validation set during the experiment.

In addition, we classified the traffic data into 17 categories based on traffic content: Aim, email, Facebook, Gmail, Hangout, ICQ, Netflix, SCP, Facebook, Spotify, Tor, Torrent, Vimeo, Buster, FTPS, SFTP, and YouTube. The specific traffic types and number of samples are listed in Table 2.

3.3. Data preprocessing

- (1) Data slicing: The original PCAP file was sliced into data at the packet level.
- (2) Data cleaning: Although the dataset was collected in a simulated real network setting, it contained useless data packets that did not lend themselves to model training. Many TCP segments included in the dataset contain SYN, ACK, or FIN flag sets, which are

critical for the three-way handshake process. However, these segments contain no information regarding the applications for which they were generated. Therefore, they must be discarded. In addition, we removed packets for DNS segments, address resolution protocols (ARP), and dynamic host configuration protocols (DHCP), which are not associated with specific traffic classifications in terms of application identification and transport content.

- (3) Uniform size: different communication sessions and the number and size of network packets transmitted vary, but the input form of the network structure is fixed; therefore, it is necessary to unify the size of the traffic. For the purposes of this study, 64 bytes were chosen as the fixed length of traffic, for a length of less than 64 bytes of data after the cutoff was filled with 0 × 00 at the end, and for a length of more than 64 bytes of data was truncated.
- (4) Marking samples: The same type of traffic data was organized and labeled.

3.4. Evaluation indicators

To comprehensively analyze the effectiveness of the classification model, we evaluated its effectiveness from the standpoints of accuracy, precision, recall, and F1. TP , TN , FP , and FN refer to the true positives, true negatives, false positives, and false negatives, respectively.

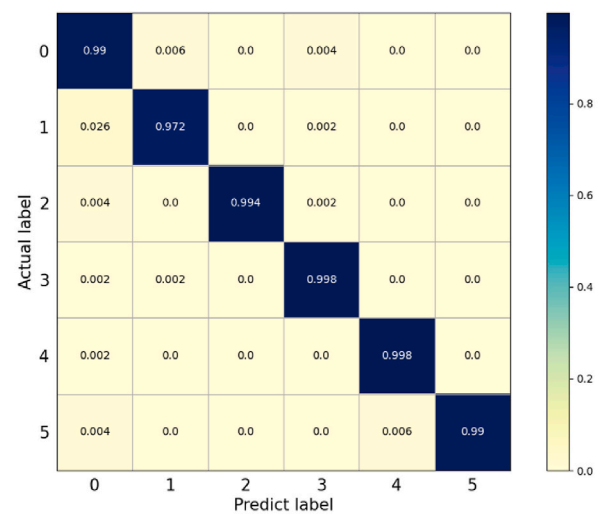
Equations (17)–(20) denote the Accuracy, Precision, Recall, and F1 values, respectively.

$$Accuracy = \frac{TP + TN}{FP + FN + TP + TN} \quad (17)$$

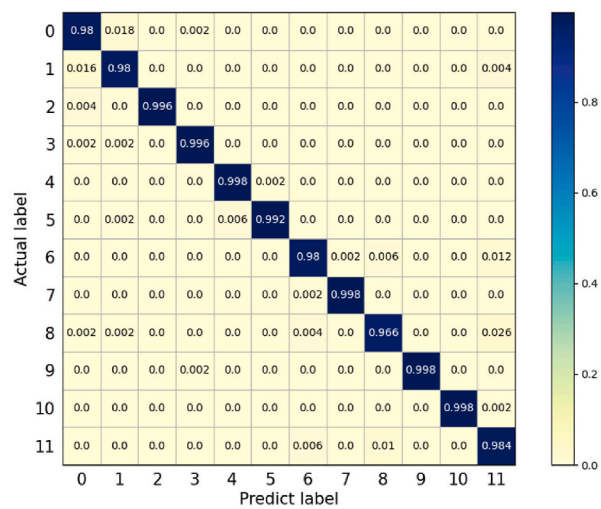
$$Precision = \frac{TP}{TP + FP} \quad (18)$$

$$Recall = \frac{TP}{TP + FN} \quad (19)$$

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall} \quad (20)$$



(b) Confusion matrix of scene 2



(d) Confusion matrix for scene 4.

Table 5
Comparison of service identification task results of ISCX VPN-nonVPN dataset.

	BCFNet			Deep Packet [24]		
Type of flow	Precision	Recall	F1-Score	Precision	Recall	F1-Score
Chat	0.980	0.976	0.978	0.84	0.71	0.77
Email	0.980	0.976	0.978	0.96	0.87	0.91
File	0.996	1.000	0.998	0.98	1.00	0.99
P2P	0.996	0.996	0.996	1.00	1.00	1.00
Streaming	0.998	0.994	0.996	0.92	0.87	0.90
Voip	0.992	0.997	0.995	0.63	0.88	0.74
VPN-Chat	0.980	0.988	0.984	0.98	0.98	0.98
VPN-Email	0.998	0.998	0.998	0.99	0.98	0.99
VPN-File	0.965	0.984	0.974	0.99	0.99	0.99
VPN-P2P	0.998	1.000	0.998	1.00	1.00	1.00
VPN-Streaming	0.998	1.000	0.999	1.00	1.00	1.00
VPN-Voip	0.984	0.957	0.970	0.99	1.00	1.00

	BCFNet			Deep Packet [24]		
Type of flow	Precision	Recall	F1-Score	Precision	Recall	F1-Score
AIM	0.97	0.985	0.977	0.87	0.76	0.81
Email	0.998	0.992	0.995	0.97	0.82	0.89
Facebook	0.986	0.992	0.989	0.96	0.95	0.96
Gmail	0.994	0.982	0.988	0.97	0.95	0.96
Hangout	0.992	0.998	0.995	0.96	0.98	0.97
ICQ	0.963	0.952	0.958	0.72	0.80	0.76
Netflix	0.994	1.000	0.997	1.00	1.00	1.00
SCP	0.994	0.994	0.994	0.97	0.99	0.98
Skype	0.996	0.994	0.995	0.94	0.99	0.97
Spotify	0.998	0.998	0.998	0.98	0.98	0.98
Tor	1.000	1.000	1.000	1.00	1.00	1.00
Torrent	0.990	0.994	0.992	1.00	1.00	1.00
Vimeo	0.996	0.996	0.996	0.99	0.99	0.99
Buster	0.992	0.994	0.993	0.99	1.00	0.99
FTPS	1.000	1.000	1.000	1.00	1.00	1.00
SFTP	0.998	1.000	0.999	1.00	1.00	1.00
Youtube	1.000	0.992	0.996	0.99	0.99	0.99

3.5. Parameter analysis experiment

When training a neural network, its weights are adjusted according to the gradient of the loss function, and the learning rate determines the magnitude of this adjustment. As shown in Fig. 4, when the learning rate is $1e-4$, the high learning rate may make the parameter update too drastic, causing the model to jump too far in the parameter space and thus miss the optimal solution with large oscillations in its loss values at epochs approximately equal to 4 and 6. By contrast, when the learning rate was $1e-6$, the learning rate was low, and the training speed of the model became slower, requiring more iterations to converge. The learning rate is too low, which also causes the model to become stuck in the local optimum conditions during the training process and cannot jump out. When the learning rate was $5e-5$, the model converged the fastest. The loss value of the model tended to stabilize after four epochs of training, and its convergence effect was significantly better than those of $1e-4$ and $1e-6$.

In a network flow, the length of packets varies greatly. Wang et al. [27] did experiments related to the effect of packet length on the performance of the system, in which the range of values of the packets was positioned in the range of 100–1000. Similar results were achieved with packet lengths of 100, 300, and 500 packets. Considering training efficiency, a packet length of 100 was chosen. As listed in Table 3, we set the packet lengths to 8, 16, 32, 64, and 128 to explore the optimal packet length further. According to Table 3, the highest accuracy was achieved in both dataset scenarios when the packet length was 64. Therefore, the packet length was set to 64 in this experiment.

3.6. Ablation experiment

The one-dimensional convolution module designed in this study consists of parallel combinations of 1×3 , 1×5 , and 1×7 convolutions, with two 1×1 convolutions serially connected in each branch. To reflect the advantages of multiscale feature fusion, we conducted six sets of ablation experiments using the ISCX VPN-nonVPN dataset, as shown in Fig. 5, in which 1×3 convolutions were used in MD1CNN. Similarly, a 1×5 convolution is used in MD2CNN, and a 1×7 convolution is used in MD3CNN.

As shown in Fig. 6, the MDT4CNN network structure module consisted of 1×3 and 1×5 convolutions. Similarly, the MDT5CNN network structure module consists of 1×3 and 1×7 convolutions, and the MDT6CNN network structure module consists of 1×5 and 1×7 convolutions.

As shown in Fig. 7, our network module consisted of 1×3 , 1×5 , and 1×7 convolutional fusions.

Based on the above six groups of network module structures, to prove the advantage of multiscale fusion in a one-dimensional convolutional module, this study conducted comparison experiments under the four indices of accuracy, precision, recall, and F1-Score (the results are shown in Table 4).

3.7. Comparison experiment

Fig. 8 shows the accuracy, precision, recall, and F1 values of each model in the four experimental scenarios of the ISCX VPN-nonVPN dataset.

As shown in Fig. 8, all the models can effectively solve the binary classification problem, with all their metrics being higher than 99%. In all the scenarios, the LSTM [28,29] model has the worst performance results because it captures the long-term dependencies in sequential data. However, in encrypted traffic, there are cases where long-term dependencies are not obvious, in which case the advantage of LSTM cannot be utilized, which leads to a poor classification effect. In scenarios 2, 3, and 4, the proposed BCFNet model outperformed the other models in all metrics. In the most complex 12 classification scenarios, the accuracy of the BCFNet model reached 98.88%, which was an

improvement of 0.54%, 0.25%, 0.97%, 9.65%, and 1.22% compared to the ET-BERT, EB-LsTm, CovLsTm, LSTM, and IM-CNN models, respectively. Compared with these classical deep learning models, this study presents a model that further improves the classification performance of the classifiers proposed in previous studies.

Fig. 9 shows the F1 metrics of the three models on the ISCX VPN-nonVPN dataset, where the proposed BCFNet model achieved the best F1 value overall in the classification results of the 12 categories of data. These two classification algorithms do not achieve the best results for the algorithm proposed in this paper for P2P and streaming among the 12 categories of data categorization. However, the results of this study's algorithm do not differ significantly from those of the optimal algorithms in the F1 results of these categories. In the P2P and Streaming categories, 1D-CNN achieves the best results. In the Chat and Email categories, the BCFNet model showed a significant improvement compared with IM-CNN and ET-BERT, and its F1 values were improved by 4.82% and 5.02% compared with IM-CNN and 4.28% and 4.26% compared with ET-BERT, respectively.

To observe more intuitively the classification accuracy of the BCFNet model for different traffic types in each scenario, as shown in Fig. 10, we plotted the heat map of the confusion matrix, which demonstrates the confusion matrix of the model under four scenarios, from the heat map of the confusion matrix. It can be seen that BCFNet can realize the binary classification accurately and without any error, that is to say, the misclassification will only occur in the categories 6_VPN and 6_nonVPN. There will not be VPN encrypted traffic incorrectly classified as a nonVPN category, which also indicates that there is a difference between the protocols used by VPN encrypted traffic and normal encrypted traffic; the accuracy of 6-classification in Scenario 2 for the nonVPN dataset is slightly higher than that of 6-classification in Scenario 3 for the VPNs, which is found in further analysis. This is because in the VPN dataset, the model confuses between File-VPN and Voip-VPN traffic, and 3.4% of File-VPN traffic is incorrectly identified as Voip-VPN. Finally, in the more complex scenario IV, each traffic category had a darker blue color, and the lowest prediction probability was 96.6%, further confirming the feasibility of the proposed model for encrypted traffic identification.

A comparison between our model and DeepPacket [30] for each class in the ISCX VPN-nonVPN and ISCX VPN-APP tasks is presented in Tables 5 and 6. DeepPacket uses only a one-dimensional convolutional neural network to extract the local features of encrypted traffic. Under the ISCX-VPN-nonVPN task, the proposed model improved by 5.89%, 4.88%, and 4.88% over the Deep Packet model in precision, recall, and F1 values, respectively, and by 1.19%, 1.18%, and 1.18%, respectively, under the ISCX VPN-Application task. This further proves that the proposed model exhibits superior performance.

4. CONCLUSION

Most previous encrypted traffic detection methods have used only ordinary CNN with a fixed-size convolutional kernel and a relatively small number of convolutional layers, which tend to lose key information when processing traffic data. In addition, the CNN learns only the local features in the packet, ignoring the relative positions between strings and the relationship between the local features and the entire packet. To address the complex characteristics of encrypted traffic, this study establishes an encrypted traffic identification method that combines multiscale features. The improved inception one-dimensional convolutional network structure is utilized to capture the local spatial features of encrypted traffic in a more comprehensive way. Second, the correlation between the bytes of the packet is captured by the MBM and SBP methods of the ET-BERT model to extract the global features of the encrypted traffic, and the fusion of the features is utilized to improve the accuracy of the encrypted traffic classification and generalization ability of the environment.

In this study, encrypted traffic was investigated from the perspective

of deep learning, and significant results were obtained. Because it is difficult to interpret the traffic features extracted by neural networks, the typical features of different types of traffic are subsequently discovered through the visualization of different network layers, such as clustering [31]. This study was primarily designed for classification accuracy and constructed a complex classification model for encrypted traffic classification. However, they did not consider practical application-deployment problems. It is of practical significance to study the balance between the accuracy and magnitude of the model. Subsequently, in-depth research was conducted from the perspectives of model interpretability and magnitude, and knowledge distillation was introduced to reduce model latency and compress network parameters to further enhance the model's capability in practical applications. The actual traffic situation is complex, the classification needs are different, and most of the existing methods are only applicable to specific needs. Future research will further explore the combination of different methods to exploit their respective advantages in migration learning.

CRedit authorship contribution statement

Peng Zhu: Writing – review & editing, Writing – original draft, Methodology, Conceptualization. **Gang Wang:** Writing – review & editing, Supervision, Resources, Investigation. **Jingheng He:** Writing – original draft, Data curation. **Yueli Dong:** Writing – original draft, Funding acquisition, Formal analysis. **Yu Chang:** Writing – original draft, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

REFERENCES

- [1] Xu C, Zhao W, Zhao J, et al. Uncertainty-aware multiview deep learning for internet of things applications. *IEEE Trans Ind Inf* 2022;19(2):1456–66.
- [2] Jia Y, Gu Z, Jiang Z, et al. Persistent graph stream summarization for real-time graph analytics[J]. *World Wide Web*; 2023. p. 1–21.
- [3] Moore AW, Papagiannaki K. Toward the accurate identification of network applications[C]. In: *International workshop on passive and active network measurement*. Berlin, Heidelberg: Springer; 2005. p. 41–54.
- [4] Ghosh A, Senthilrajan A. Classifying network traffic using DPI and DFL. *International journal of scientific and technology research* 2019;8(11):1019.
- [5] Bi X, Nie H, Zhang G, et al. Boosting question answering over knowledge graph with reward integration and policy evaluation under weak supervision. *Inf Process Manag* 2023;60(2):103242.
- [6] Zhang D, Jin K. Fast algorithms for computing the statistics of pattern matching. *IEEE Access* 2021;9:114965–76.
- [7] Duan Y, Long H, Qu YQ. Application of improved BM algorithm in string approximate matching. *Procedia Comput Sci* 2020;166:576–81.
- [8] Trivedi U. An optimized Aho-corasick multi-pattern matching algorithm for fast pattern matching[C]//2020 IEEE 17th India council international conference (INDICON). IEEE; 2020. p. 1–5.
- [9] Zhao S, Xiao Y, Ning Y, et al. An optimized K-means clustering for improving accuracy in traffic classification. *Wireless Pers Commun* 2021;120(1):81–93.
- [10] Saber A, Belkacem F, Moncef A. Encrypted network traffic identification: LDA-KNN approach[C]. In: *Proceedings of the 9^{ème} édition du colloque Tendances dans les Applications Mathématiques en Tunisie Algérie et Maroc*. Tlemcen, Algeria; 2019. p. 23–7.
- [11] Ke Xin Lv. Research on network encrypted traffic classification based on deep learning[D]. Shanghai Normal University 2021. <https://doi.org/10.27312/d.cnki.gshsu.000791>.
- [12] Chen Xuejiao, Wang Pan, Yu Jiahui. An encrypted traffic identification method based on convolutional neural network. *Journal of Nanjing University of Posts and Telecommunications (Natural Science Edition)* 2018;38(6):36–41. <https://doi.org/10.14132/j.cnki.1673-5439.2018.06.006>.
- [13] Chen MH, Zhu YF, Lu B, Zhai Y, Li D. Attention-CNN-based application type recognition for encrypted traffic. *Computer Science* 2021;48(4):325–32.
- [14] Liu C, He L, Xiong G, et al. Fs-net: a flow sequence network for encrypted traffic classification[C]//IEEE INFOCOM 2019-IEEE Conference on Computer Communications. IEEE; 2019. p. 1171–9.
- [15] Zou Yuan, Zhang A, Jiang Bin. Detection of malicious encrypted traffic based on LSTM recurrent neural network. *Computer Application and Software* 2020;37(2): 308–12.
- [16] Wang W. Research on network traffic classification and anomaly detection method based on deep learning [D]. University of Science and Technology of China; 2018.
- [17] Wang Qinfan, Zhai Jiangtao, Chen Wei, Sun Haoxiang. An encrypted traffic classification method based on graph convolutional neural network. *Electronic Measurement Technology* 2022;45(14):109–15.
- [18] Fang U, Li J, Lu X, et al. Robust image clustering via context-aware contrastive graph learning. *Pattern Recogn* 2023;138:109340.
- [19] Yao H, Liu C, Zhang P, et al. Identification of encrypted traffic through attention mechanism based long short term memory[J]. *IEEE Transactions on Big Data*; 2019.
- [20] Kim TY, Cho SB. Web traffic anomaly detection using C-LSTM neural networks. *Expert Syst Appl* 2018;106:66–76.
- [21] Lin X, Xiong G, Gou G, et al. Bert ET. A contextualized datagram representation with pre-training transformers for encrypted traffic classification[C]. *Proceedings of the ACM Web Conference 2022*;2022:633–42.
- [22] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. *Adv Neural Inf Process Syst* 2017;30.
- [23] Ioffe S, Szegedy C. Batch normalization: accelerating deep network training by reducing internal covariate shift[C]//International conference on machine learning. pmlr; 2015. p. 448–56.
- [24] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016. p. 770–8.
- [25] Draper-gil G, Lashkari AH, Mamun MSI, et al. Characterization of encrypted and vpn traffic using time-related[C]//Proceedings of the 2nd international conference on information systems security and privacy. ICISSP; 2016. p. 407–14.
- [26] Zhang Shurong, Bu Youjun, Chen Bo, et al. An encrypted traffic classification method based on multilayer bi-directional SRU with attention model. *Comput Eng* 2022;48(11):127–36. <https://doi.org/10.19678/j.issn.1000-3428.0063626>.
- [27] Wang W. Research on network traffic classification and anomaly detection method based on deep learning [D]. University of Science and Technology of China; 2018.
- [28] Redhu P, Kumar K. Short-term traffic flow prediction based on optimized deep learning neural network: PSO-Bi-LSTM. *Phys Stat Mech Appl* 2023;625:129001.
- [29] Liu J, Chen Y, Huang X, et al. GNN-based long and short term preference modeling for next-location prediction. *Inf Sci* 2023;629:1–14.
- [30] Lotfollahi M, Jafari Siavoshani M, Shirali Hossein Zade R, et al. Deep packet: a novel approach for encrypted traffic classification using deep learning. *Soft Comput* 2020;24(3):1999–2012.
- [31] Fang U, Li M, Li J, et al. A comprehensive survey on multi-view clustering[J]. *IEEE Transactions on Knowledge and Data Engineering*; 2023.